

Documentação do Projeto: MinRoute

Sistema de Navegação Primitivo baseado no Algoritmo de Dijkstra

INF/UFG 2026-1-AED2

Prof. André L. Moura

5 de maio de 2026

Integrantes

- (Nome completo em ordem alfabética)
- (Nome completo em ordem alfabética)
- (Nome completo em ordem alfabética)
- (Nome completo em ordem alfabética)

Conteúdo

1	Introdução	3
1.1	Objetivo do Documento	3
1.2	Escopo do Projeto e Objetivos	3
1.3	Visão Geral do Sistema	3
2	Arquitetura do Sistema	3
2.1	Visão Arquitetural	3
2.2	Descrição das Camadas	3
2.2.1	Camada de Apresentação (UI)	3
2.2.2	Camada de Domínio (Core)	3
2.2.3	Módulos Utilitários	3
3	Estrutura do Projeto	4
4	Detalhamento dos Módulos	4
4.1	Módulo Core ('src/core')	4
4.2	Módulo UI ('src/ui')	4
5	Funcionalidades Detalhadas	4
5.1	Requisitos Funcionais (RF)	4
5.2	Requisitos Não Funcionais (RNF)	5
6	Formatos de Dados de Entrada	5
6.1	Arquivo '.txt' (Lista de Adjacência)	5
6.2	Arquivo '.osm' (OpenStreetMap)	5
6.3	Arquivo '.poly'	5
7	Guia de Instalação e Execução	6
7.1	Pré-requisitos	6
7.2	Passos para Instalação	6
8	Guia de Uso da Aplicação	6
8.1	Interação Visual	6
8.2	Encontrar Caminho Mínimo	6
8.3	Manipulação e Exportação	6

1 Introdução

1.1 Objetivo do Documento

Este documento apresenta a especificação técnica, arquitetural e funcional do sistema MinRoute, desenvolvido como projeto final da disciplina de Algoritmos e Estruturas de Dados 2 (AED2). O sistema visa encontrar o caminho mais curto entre dois pontos geográficos utilizando grafos e o algoritmo de Dijkstra.

1.2 Escopo do Projeto e Objetivos

O NavGraph é um sistema com interface visual que permite importar mapas reais, convertê-los em grafos e calcular rotas indicativas. Os objetivos principais incluem o uso eficiente de estruturas de dados (Lista de Adjacência e Heap Mínima) para garantir alto desempenho em grafos com milhares de vértices.

1.3 Visão Geral do Sistema

O software é composto por um motor de cálculo (Módulo Core) e uma interface interativa com design minimalista (Módulo UI). O sistema processa arquivos de mapas (.osm, .poly, .txt) para visualização e manipulação em tempo real de redes viárias locais.

2 Arquitetura do Sistema

2.1 Visão Arquitetural

A arquitetura é modular e baseada em camadas, separando a lógica pesada de negócio da camada de interação visual. A comunicação entre os ecossistemas ocorre de forma invisível via execução de subprocessos locais, realizando a troca de dados geográficos e matemáticos estruturados em JSON.

2.2 Descrição das Camadas

2.2.1 Camada de Apresentação (UI)

Desenvolvida com foco em uma experiência de usuário limpa, é responsável pela renderização do grafo no canvas, gestão de eventos de mouse e exibição de estatísticas. Utiliza tipografia de alta legibilidade (JetBrains Mono) nos painéis de resposta para garantir clareza nas métricas de execução.

2.2.2 Camada de Domínio (Core)

Contém a lógica matemática estrita. Responsável por traduzir os polígonos em nós na memória e garantir o tempo de resposta otimizado através da gestão espacial da Fila de Prioridade.

2.2.3 Módulos Utilitários

Scripts auxiliares focados em pontes de comunicação entre linguagens, formatação de saída e rotinas de exportação de imagem para a área de transferência.

3 Estrutura do Projeto

Para facilitar a manutenção e manter a separação de responsabilidades (Frontend e Backend), a base de código do projeto foi estruturada da seguinte forma:

```
MinRoute/
|-- Documentacao/
|   '-- DocumentacaoDoProjeto.pdf
|-- data/
|   '-- campus2ufg&regiao.poly
|-- src/
|   |-- core/ (Java)
|   |   |-- DijkstraCore.java
|   |   |-- Grafo.java
|   |   '-- ParserPoly.java
|   |-- ui/ (Python)
|   |   |-- app.py
|   |   |-- canvas_mapa.py
|   |   '-- painel_estatisticas.py
|   '-- utils/
|       |-- comunicacao_core.py
|       '-- exportar_imagem.py
|-- build/
|-- dist/
|-- instalador_script.iss
'-- Read-me.txt
```

4 Detalhamento dos Módulos

4.1 Módulo Core ('src/core')

O motor de cálculo utiliza uma **Lista de Adjacência** para garantir um armazenamento esparsa e eficiente em memória de $O(V + E)$. Em conjunto, implementa-se uma **Heap Mínima** para a extração do menor custo no algoritmo de Dijkstra, assegurando complexidade de tempo de $O(E \log V)$.

4.2 Módulo UI ('src/ui')

Interface desenhada para evitar sobrecarga visual, isolando as ferramentas de edição e focando na exploração do grafo. A conversão das coordenadas espaciais é gerida nativamente pelo componente de Canvas, mapeando os pontos originais para os pixels em tela.

5 Funcionalidades Detalhadas

5.1 Requisitos Funcionais (RF)

- **RF01:** Importação de mapas reais (.txt, .xml, .osm, .poly) e conversão para grafos em memória.

- **RF02:** Enumeração de vértices e rotulação das arestas baseada em distâncias euclidianas.
- **RF03:** Seleção interativa de origem e destino, representados por marcações de cores distintas.
- **RF04:** Cálculo em segundo plano e exibição instantânea da rota indicativa do menor caminho.
- **RF05:** Criação e deleção manual de vértices e arestas através do cursor.
- **RF06:** Lógica de suporte a grafos direcionados e não direcionados (fluxos de mão única e dupla).
- **RF07:** Painel lateral de métricas para exibir tempo de processamento, número de nós explorados e custo.
- **RF08:** Botão para capturar a imagem atual do grafo e enviá-la para a área de transferência do sistema.

5.2 Requisitos Não Funcionais (RNF)

- **RNF01 e RNF02:** Paleta de cores rigorosa para diferenciar vértices, arestas de mão dupla e vias de sentido único.
- **RNF03 e RNF05:** Operação estável e econômica em memória sob estresse mecânico de grafos com múltiplos milhares de vértices.
- **RNF04:** Resolução de cálculos complexos e tempo total de resposta de ponta a ponta inferior a 2 segundos para instâncias médias.
- **RNF06:** Acessibilidade e navegação fluida, garantindo que o programa seja operável por usuários não familiarizados com grafos.
- **RNF07 e RNF08:** Código extensível e compilável tanto em instâncias de Windows nativo quanto em ecossistemas de terminal Linux.

6 Formatos de Dados de Entrada

6.1 Arquivo '.txt' (Lista de Adjacência)

Entrada legível por humanos para testes de mesa simplificados.

6.2 Arquivo '.osm' (OpenStreetMap)

Malha crua georreferenciada exportada em XML, mantendo coordenadas de latitude e longitude originais.

6.3 Arquivo '.poly'

Formato transformado através da projeção Universal Transversa de Mercator (UTM), já compatibilizando a escala com o grid cartesiano, acelerando radicalmente o tempo de plotagem e processamento.

7 Guia de Instalação e Execução

7.1 Pré-requisitos

- Computador rodando Windows (10/11) ou ambiente isolado WSL (Ubuntu) para modo de desenvolvimento.
- JRE 11+ para processamento das classes de cálculo.

7.2 Passos para Instalação

Para a avaliação final, execute o instalador principal gerado pelo Inno Setup (`Setup.exe`). Os binários do Python e o arquivo compilado em ‘.jar’ serão alocados localmente junto com a base de dados cartográfica.

8 Guia de Uso da Aplicação

8.1 Interação Visual

O Canvas principal suporta navegação nativa. Elementos que representam rodovias ou calçadas são renderizados no carregamento do módulo.

8.2 Encontrar Caminho Mínimo

Com o mapa aberto, um clique esquerdo define o ponto inicial (S) de análise. Um segundo clique registra o destino alvo (T). A rota ótima é traçada visualmente de maneira instantânea.

8.3 Manipulação e Exportação

Ferramentas da barra superior gerenciam a importação manual de arquivos de texto brutais, além de autorizar a limpeza da interface e exportação das visualizações para o *clipboard*.